

Revisão de Álgebra Linear

Inversão de Matrizes e Solução de Sistemas Lineares
Aplicação ao módulo de estimação de parâmetros do motor (SMMA)

Estudo de apoio – 07.Studies/inversao_de_matrizes

Agosto de 2026

1 O problema que estamos resolvendo

Quando o SMMA monta um “conjunto de equações lineares obtidas com os dados dos sensores”, isso significa um sistema do tipo:

$$A \cdot x = b \quad (1)$$

onde:

- A é uma matriz de coeficientes (dimensão máxima 4×4) – vem do modelo do motor (ex.: relações entre vibração, corrente, temperatura e os parâmetros físicos que se quer estimar);
- x é o vetor de incógnitas – os parâmetros do motor que queremos estimar;
- b é o vetor de valores conhecidos – medidos pelos sensores.

A pergunta central da álgebra linear aqui é: dado A e b , como encontrar x ?

2 Quando existe solução – e quando ela é única

Um sistema quadrado (n equações, n incógnitas) tem solução **única** se e somente se A é **não singular**, ou seja:

$$\det(A) \neq 0 \quad (2)$$

Se $\det(A) = 0$, a matriz é singular: ou não há solução, ou há infinitas. Fisicamente, isso corresponde a equações “redundantes” ou “conflitantes” vindas dos sensores – é exatamente por isso que o descritivo do projeto pede tratamento para “pivô igual ou próximo de zero”: um pivô perto de zero é sintoma de uma matriz perto de ser singular (mal condicionada), mesmo que o determinante não seja exatamente zero.

3 O que é a matriz inversa e por que ela resolve o sistema

Se A é não singular, existe uma matriz A^{-1} tal que:

$$A \cdot A^{-1} = A^{-1} \cdot A = I \quad (\text{matriz identidade}) \quad (3)$$

Multiplicando os dois lados de $A \cdot x = b$ por A^{-1} pela esquerda:

$$\begin{aligned} A^{-1} \cdot A \cdot x &= A^{-1} \cdot b \\ I \cdot x &= A^{-1} \cdot b \\ x &= A^{-1} \cdot b \end{aligned} \quad (4)$$

Ou seja: se tivermos A^{-1} , resolvemos qualquer sistema com aquele A apenas multiplicando por b – útil quando o mesmo A é reutilizado com vários b diferentes. No SMMA, a cada janela de amostras (a cada 10 ms), A pode mudar pouco, mas o vetor b certamente muda a cada nova amostra.

Nota conceitual. Em software numérico “sério”, quase nunca se calcula A^{-1} explicitamente para resolver um único sistema – resolve-se diretamente por eliminação, porque é mais barato e mais estável numericamente. O motivo de precisarmos da inversa **explícita** neste projeto é que o enunciado pede os parâmetros estimados como resultado reutilizável/feature para o classificador de Machine Learning, e não apenas a solução pontual de uma equação.

4 Como os algoritmos calculam isso, na prática

Todos os métodos a seguir respondem à mesma pergunta – *como chegar em x (ou em A^{-1}) a partir de A e b* – mas atacam o problema de formas diferentes.

4.1 Eliminação de Gauss-Jordan

A ideia é aplicar operações elementares sobre linhas (trocar linhas, multiplicar uma linha por escalar, somar múltiplo de uma linha a outra) simultaneamente em A e na matriz identidade, até A virar I :

$$[A \mid I] \xrightarrow{\text{operações de linha}} [I \mid A^{-1}] \quad (5)$$

Passo a passo, para cada coluna $k = 1, \dots, n$:

1. **Escolher o pivô:** idealmente o elemento de maior módulo na coluna k (entre as linhas ainda não processadas) – isso é o *pivotamento parcial*, que evita dividir por números pequenos.
2. **Trocar linhas** se necessário, para trazer esse pivô à posição (k, k) .
3. **Normalizar a linha do pivô:** dividir toda a linha pelo valor do pivô, de modo que a posição (k, k) passe a valer 1.
4. **Zerar a coluna k em todas as outras linhas:** para cada outra linha i , subtrair (elemento $_{i,k} \times$ linha do pivô) – isso cria zeros acima e abaixo do pivô simultaneamente, o que distingue Gauss-Jordan da eliminação de Gauss simples.

No final, o lado que começou como I torna-se A^{-1} .

Por que é caro em hardware: cada uma das n etapas de pivô realiza uma divisão (passo 3) e aproximadamente n^2 multiplicações/subtrações (passo 4). Para 4×4 , isso resulta em uma estrutura de laços bem previsível – favorável para uma implementação em FPGA, que exige laços com limites determináveis em tempo de síntese.

4.2 Decomposição LU

A ideia é fatorar $A = L \cdot U$, onde L é triangular inferior (com 1's na diagonal) e U é triangular superior. Essa fatoração essencialmente “congela” o processo de eliminação de Gauss em duas matrizes, em vez de aplicá-lo diretamente sobre b .

Com L e U em mãos, resolver $A \cdot x = b$ se transforma em resolver dois sistemas triangulares, que são baratos computacionalmente:

$$\begin{aligned}
& L \cdot U \cdot x = b \\
& \text{fazendo } y = U \cdot x : \quad L \cdot y = b \quad (\text{substituição direta – } forward \text{ substitution}) \\
& \quad \quad \quad U \cdot x = y \quad (\text{substituição reversa – } backward \text{ substitution}) \quad (6)
\end{aligned}$$

Para obter a **inversa explícita** (não apenas resolver um sistema), repete-se esse processo n vezes, usando como b cada coluna da identidade e_1, e_2, e_3, e_4 – cada solução obtida vira uma coluna de A^{-1} .

Vantagem real de LU: se A muda pouco mas é necessário resolver para vários b (ou reconstruir a inversa periodicamente), fatorar uma única vez e reaproveitar L e U é mais barato do que refazer Gauss-Jordan do zero a cada vez. Apresenta o mesmo problema de pivô/divisão que Gauss-Jordan, pois é essencialmente o mesmo processo reorganizado em duas etapas.

4.3 Decomposição QR

A ideia é fatorar $A = Q \cdot R$, onde Q é ortogonal ($Q^T \cdot Q = I$) e R é triangular superior. Construída via rotações de Givens ou reflexões de Householder, que zeram elementos abaixo da diagonal sem depender de “dividir pelo pivô” – em vez disso, usam operações que envolvem raiz quadrada.

Resolução do sistema:

$$\begin{aligned}
& A \cdot x = b \\
& Q \cdot R \cdot x = b \\
& R \cdot x = Q^T \cdot b \quad (\text{pois } Q^T \cdot Q = I) \quad (7)
\end{aligned}$$

$R \cdot x = Q^T \cdot b$ é triangular e se resolve por substituição reversa, exatamente como no método LU.

Vantagem: muito mais estável numericamente – não sofre com pivôs pequenos da mesma forma, pois não há divisão por um pivô arbitrário no processo de zerar elementos. *Desvantagem para FPGA:* raiz quadrada (ou as aproximações de Householder) é cara e pouco natural em ponto fixo – normalmente representa um custo desnecessário (*overkill*) para uma matriz 4×4 razoavelmente bem-condicionada, como se espera do problema do motor.

4.4 Método de cofatores / matriz adjunta

Fórmula fechada, direta da definição:

$$A^{-1} = \frac{1}{\det(A)} \cdot \text{adj}(A) \quad (8)$$

onde $\text{adj}(A)$ (matriz adjunta) é a transposta da matriz de cofatores, e cada cofator é dado por $C_{ij} = (-1)^{i+j} \cdot M_{ij}$, com M_{ij} sendo o determinante do “menor” – a submatriz obtida removendo a linha i e a coluna j .

Para calcular o determinante de uma matriz 4×4 por expansão de Laplace, é necessário, recursivamente, obter os determinantes de submatrizes 3×3 , que por sua vez dependem de determinantes 2×2 , e assim por diante – o número de multiplicações cresce fatorialmente com n . Para $n = 4$ ainda é “aceitável” (algumas dezenas de multiplicações), mas o método não escala e não possui a estrutura regular de laços que o Gauss-Jordan apresenta.

Único cenário onde faz sentido em hardware: se houver interesse em evitar completamente a divisão durante o processo de eliminação (restando apenas uma divisão final por $\det(A)$ ao término) – pode ser um argumento técnico interessante para justificar o uso de cofatores, mas o custo em multiplicadores tende a ser maior.

5 Resumo comparativo

Aspecto	Gauss-Jordan	LU	QR	Cofatores
Operação dominante	divisão + MAC	divisão + MAC	raiz quadrada + MAC	apenas multiplicação (até o fim)
N.º de operações	$O(n^3)$	$O(n^3)$	$O(n^3)$, constante maior	$O(n!)$
Trata pivô pequeno?	precisa de pivotamento explícito	idem	naturalmente mais estável	não há “pivô”; $\det \approx 0$ quebra da mesma forma
Estrutura de laços	regular, fácil de mapear em FSM	duas fases (fatorar + 2 substituições)	mais irregular (rotações)	recursivo, irregular
Adequação a boa FPGA / ponto fixo	boa	boa	ruim (raiz quadrada)	aceitável só para 4×4 , porém caro

6 Conexão com a decisão de arquitetura do SMMA

Esta revisão é o passo teórico anterior à definição da arquitetura do módulo `matrix_inv` do acelerador. Os pontos abaixo resumem o que a teoria acima implica diretamente para o projeto de hardware:

- O tratamento de **pivô próximo de zero** não é um detalhe de implementação – é uma consequência direta de $\det(A) \approx 0$, ou seja, de o sistema físico estar mal condicionado. O mecanismo de pivotamento parcial (Seção 4.1, passos 1-2) é a resposta natural a essa pergunta do descritivo do problema.
- A escolha entre Gauss-Jordan, LU, QR e cofatores é, em essência, uma troca entre **estabilidade numérica** (QR > LU/Gauss-Jordan > cofatores) e **custo/regularidade em hardware** (Gauss-Jordan e LU > cofatores > QR).
- Para uma matriz 4×4 com bom condicionamento esperado (motor operando em regime razoavelmente estável), **Gauss-Jordan com pivotamento parcial** tende a ser o melhor equilíbrio entre simplicidade de controle (FSM regular, laços com limite determinável) e custo aritmético.
- O **formato de ponto fixo** escolhido precisa acomodar a divisão do passo de normalização (Seção 4.1, passo 3), que é o ponto mais sensível a perda de precisão/overflow ao longo das etapas de eliminação.